

CSC-3TC33

Algorithmique et fondements de l'informatique

Structures de données - partie b

Bertrand Meyer

Automne 2024

Contexte

Point de méthodologie

On parle de **barrière d'abstraction** pour désigner le fait que l'on doit pouvoir manipuler des abstractions de haut niveau sans se soucier de leurs détails d'implémentation (voire même sans les connaître!).

On dit qu'un code est **modulaire** s'il est composé d'éléments les plus indépendants possibles les uns des autres, et qui fonctionnent ensemble simplement.

Exemple

En Python, il existe des modules qui se chargent avec

import nom_module.

En C, on transmet l'information d'un fichier **.c** à travers un fichier d'entête **.h**.

Type de donnée abstrait

Définition

Un **type de donnée abstrait** est un modèle mathématique qui décrit des données ainsi qu'un ensemble d'opérations que l'on peut effectuer sur elles.

Caractéristiques

- fonctionne comme un cahier des charges (signature des primitives, axiomes, préconditions)
- usage indépendant de son implémentation concrète.
- plusieurs implémentations possibles.
- formes diverses des mécanismes d'abstraction selon le langage ou le paradigme de programmation

Respect de la barrière d'abstraction : ignorer les détails d'implémentation du type abstrait et s'obliger à toujours manipuler une donnée avec des opérations fournies par le type abstrait.

1974



Barbara Liskov
(1939)

Primitives particulières

Vocabulaire

Parmi les opérations d'un type abstrait, on qualifie

- de **constructeur** l'opération de création d'une instance du type
En Python, il s'appelle par convention `__init__`.
- d'**accesseur** les opérations d'observation ou de consultation de l'état de l'instance.
- de **mutateur** les opérations de transformation de l'état de l'instance.

NB : Dans les langages dépourvus de ramasse-miette, il faut aussi penser à libérer la mémoire allouée sur le tas avec un **destructeur**

Structure de données concrète

Définition

Une **structure de donnée** est une construction explicite qui met en œuvre un type abstrait à partir des possibilités offertes par le langage.

Exemple

Quelques mécanismes mis à profit :

En C :

- **variables** de type entier, char, etc.
- **tableaux**, i.e. suite de variables de même type et contiguës en mémoire
- **pointeurs**
- **enregistrements** (**struct**)
- **alias** (**typedef**)

En Python :

- **objets** de types préexistants (entier, liste, dictionnaire)
- **noms** reliés à des objets via leur identifiant
- **classes** pour construire de nouveaux objets

Rappel : pas d'adresse en Python, mais des identifiants.

Mutable, immuable ou persistant

Définition

Une **structure de donnée mutable** est une structure dont l'état peut évoluer au cours du temps.

Définition

Une **structure de donnée immuable** est une structure dont l'état est défini une fois pour toute.

Avantage : il est plus facile de raisonner sur un programme aux données immuables (particulièrement dans le cas concurrent).

Définition

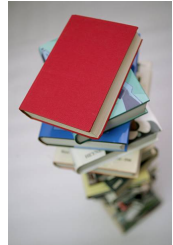
Une structure de données est **persistante** si elle préserve ses versions antérieures lorsqu'elle est modifiée; une telle structure est *immuable*.

Types abstraits et structures linéaires

Type abstrait de donnée Pile

Spécification informelle

Une **pile** est une collection qui permet de stocker des objets par ordre d'arrivée et de les extraire à partir du dernier arrivé (**Dernier arrivé, premier servi**).



Type abstrait de donnée Pile

Spécification informelle

Une **pile** est une collection qui permet de stocker des objets par ordre d'arrivée et de les extraire à partir du dernier arrivé (**Dernier arrivé, premier servi**).

Primitives souhaitées (et leur signature)

- `PileVide : () → Pile`
- `EstVide : Pile → Booléen`
- `Empiler : Pile × Elmt → Pile`
- `Dépiler : Pile → Pile × Elmt`



Type abstrait de donnée Pile

Spécification informelle

Une **pile** est une collection qui permet de stocker des objets par ordre d'arrivée et de les extraire à partir du dernier arrivé (**Dernier arrivé, premier servi**).

Primitives souhaitées (et leur signature)

- `PileVide : () → Pile`
- `EstVide : Pile → Booléen`
- `Empiler : Pile × Elmt → Pile`
- `Dépiler : Pile → Pile × Elmt`



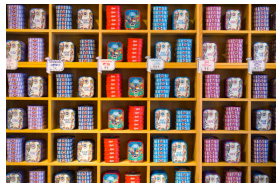
Applications

Compilateur, piles d'adresse de retour (appels récursifs de fonctions), variables locales, évaluation d'expressions arithmétiques, parcours en profondeur (appels récursifs).

Spécification formelle du type abstrait Pile

Primitives souhaitées

- $\text{PileVide} : () \rightarrow \text{Pile}$
- $\text{EstVide} : \text{Pile} \rightarrow \text{Booléen}$
- $\text{Empiler} : \text{Pile}, \text{Element} \rightarrow \text{Pile}$
- $\text{Depiler} : \text{Pile} \rightarrow \text{Pile}, \text{Element}$



Modélisation par préconditions & règles

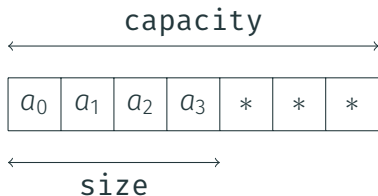
- EstVide satisfait à

$$\text{EstVide}(p) = \begin{cases} \text{Vrai} & \text{si } p = \text{PileVide}() \\ \text{Faux} & \text{si } p \text{ est de la forme } \text{Empiler}(p', x) \end{cases}$$

- Depiler n'est valide que si $\text{EstVide}(p)$ vaut Faux et alors

$$\text{Depiler}(p) = (p', x) \text{ où } p \text{ est de la forme } \text{Empiler}(p', x)$$

Implémentation en C par un tableau



```
typedef struct mystack
↳ mystack_t;
struct mystack {
    int capacity;
    int size;
    // 0 ≤ size ≤ capacity
    int *data;
    // tableau de taille capacity
};
```

Type abstrait de donnée File

Spécification informelle

Une **file** est une collection qui permet de stocker des objets par ordre d'arrivée et de les extraire à partir du premier arrivé (**Premier arrivé, premier servi**).



Type abstrait de donnée File

Spécification informelle

Une **file** est une collection qui permet de stocker des objets par ordre d'arrivée et de les extraire à partir du premier arrivé (**Premier arrivé, premier servi**).

Primitives souhaitées (et leur signature)

- `FileVide : () → File`
- `EstVide : File → Booléen`
- `Enfiler : File, Element → File`
- `Defiler : File → File, Element`



Type abstrait de donnée File

Spécification informelle

Une **file** est une collection qui permet de stocker des objets par ordre d'arrivée et de les extraire à partir du premier arrivé (**Premier arrivé, premier servi**).

Primitives souhaitées (et leur signature)

- `FileVide : () → File`
- `EstVide : File → Booléen`
- `Enfiler : File, Element → File`
- `Defiler : File → File, Element`



Applications

Routeurs, gestion d'impressions, tampons (flux de données à traiter), parcours en largeur.

Spécification formelle du type abstrait File

Primitives souhaitées

- $\text{FileVide} : () \rightarrow \text{File}$
- $\text{EstVide} : \text{File} \rightarrow \text{Booléen}$
- $\text{Enfiler} : \text{File}, \text{Element} \rightarrow \text{File}$
- $\text{Defiler} : \text{File} \rightarrow \text{File}, \text{Element}$



Modélisation par préconditions & règles

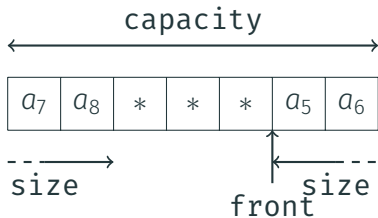
- EstVide satisfait à

$$\text{EstVide}(f) = \begin{cases} \text{Vrai} & \text{si } f = \text{FileVide}() \\ \text{Faux} & \text{si } f \text{ est de la forme } \text{Enfiler}(f', x) \end{cases}$$

- Defiler n'est valide que si $\text{EstVide}(f)$ vaut Faux et alors :

$$\text{Defiler}(\text{Enfiler}(f, x)) = \begin{cases} \text{FileVide}(), x & \text{si } f = \text{FileVide}() \\ \text{Enfiler}(g, x), y & \text{sinon, où } (g, y) = \text{Defiler}(f). \end{cases}$$

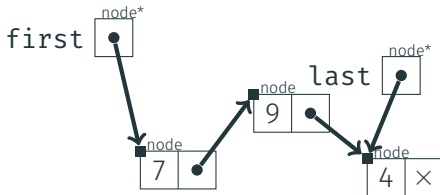
Implémentation en C par un tableau



Éléments stockés du plus ancien
au plus récent entre l'indice
front inclus et l'indice
front+size exclu.

```
typedef struct myqueue myqueue_t;  
struct myqueue {  
    int capacity;  
    int front;  
    int size;  
    int *data;  
};
```

Implémentation en C par des maillons chaînés



Éléments stockés du plus ancien
au plus récent entre le maillon
first et le maillon **last**.

```
typedef struct node node_t;
struct node {
    int data;
    node_t *next;
};

typedef struct myqueue myqueue_t;
struct mystack {
    node_t *first;
    node_t *last;
};
```

Spécification informelle

Une **deque** ou file d'attente à double extrémité (en anglais *double ended queue*) est une collection séquentielle qui permet d'ajouter et d'extraire les éléments à l'avant et à l'arrière.

Type de donnée abstrait Deque

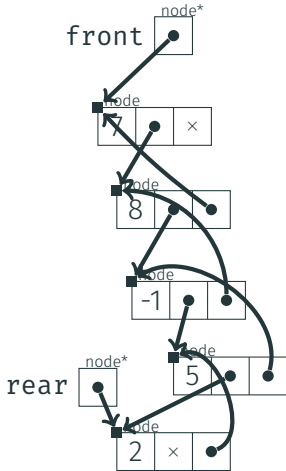
Spécification informelle

Une **deque** ou file d'attente à double extrémité (en anglais *double ended queue*) est une collection séquentielle qui permet d'ajouter et d'extraire les éléments à l'avant et à l'arrière.

Primitives souhaitées (et leur signature)

- `DequeVide : () → Deque`
- `EstVide : Deque → Booléen`
- `Enfiler_avant : Deque, Element → Deque`
- `Defiler_avant : Deque → Deque, Element`
- `Enfiler_arrière : Deque, Element → Deque`
- `Defiler_arrière : Deque → Deque, Element`

Implémentation en Python : rien à faire



```
from collections import deque
```

```
d = deque()
```

```
d.append(-1)
```

```
d.append(5)
```

```
d.appendleft(8)
```

```
d.append(2)
```

```
d.appendleft(7)
```

```
d.popleft()
```

```
d.pop()
```

Les deque sont commodes pour faire des **files** en Python!

Existent aussi en C++ ou Java.

Insertion ou suppression en tête et en queue en $O(1)$.

Spécification informelle

Un **vecteur** ou **tableau** redimensionnable est une collection séquentielle qui permet l'accès direct au i -ième élément et l'insertion d'éléments en fin de tableau.

Type de donnée abstrait Vecteur

Spécification informelle

Un **vecteur** ou **tableau** redimensionnable est une collection séquentielle qui permet l'accès direct au i -ième élément et l'insertion d'éléments en fin de tableau.

Primitives souhaitées (et leur signature)

- `VecteurVide : () → Vecteur`
- `EstVide : Vecteur → Booléen`
- `Empiler : Vecteur, Element → Vecteur`
- `Depiler : Vecteur → Vecteur, Element`
- `Consulter : Vecteur, Entier → Element`
- `Ecrire : Vecteur, Entier, Element → Vecteur`

Implémentation en C par un tableau

On utilise un tableau de capacité bornée 2^k .

Si la capacité est atteinte, on alloue un nouveau tableau de capacité 2^{k+1} . On recopie les éléments.

Théorème

Complexité amortie de l'insertion : $O(1)$.

Démonstration.

En effet, toutes les 2^{k-1} insertions, la capacité est atteinte. On paye alors $O(2^k)$ pour copier. En moyenne, on paye $O\left(\frac{2^k}{2^{k-1}}\right) = O(1)$. $\square$

```
typedef struct vector vector_t;
struct vector {
    int capacity;
    int size;
    // 0 <= size <= capacity
    int *data;
    // tableau de taille capacity
};
```

Implémentation en Python

Fait important

Le type `list` de Python désigne des tableaux redimensionnables.

Conséquence

La complexité en temps des opérations d'insertion en fin de liste, de consultation ou écriture d'une case est $O(1)$.

L'insertion en milieu de liste est plus coûteuse.

Pour les curieux (hors programme!) : le code original C

```
typedef struct {
    PyObject_HEAD
    Py_ssize_t ob_size;

    /* Vector of pointers to list elements.
    ↪ list[0] is ob_item[0], etc. */
    PyObject **ob_item;

    /* ob_item contains space for
    ↪ 'allocated' elements. The number
    * currently in use is ob_size.
    * Invariants:
    *     0 <= ob_size <= allocated
    *     len(list) == ob_size
    *     ob_item == NULL implies ob_size
    ↪ == allocated == 0
    */
    Py_ssize_t allocated;
} PyListObject;
```

Cf. <https://stackoverflow.com/questions/3917574/how-is-pythons-list-implemented>

Les listes en tant que type abstrait

Avertissement

En informatique, le terme **liste** est ambigu et désigne des types abstraits ou des structures différentes selon les langages.

Spécification informelle

Une **liste** est une collection qui permet de stocker des éléments de manière ordonnée avec un accès libre à tous les éléments.



Les listes en tant que type abstrait fourre-tout

Primitives plus ou moins souhaitées

- `ListeVide` : $() \rightarrow \text{Liste}$
- `EstVide` : $\text{Liste} \rightarrow \text{Booléen}$
- `Insérer` : $\text{Liste} \times \text{Elmt} \rightarrow \text{Liste}$
- `Supprimer` : $\text{Liste} \rightarrow \text{Liste} \times \text{Elmt}$
- `Premier, Dernier` : $\text{Liste} \rightarrow \text{Elmt}$
- `Queue` : $\text{Liste} \rightarrow \text{Liste}$
- `Suivant, Précédent` : $\text{Liste} \times (\text{Adr/Indice}) \rightarrow \text{Elmt}$
- `InsérerAprès` : $\text{Liste} \times (\text{Adr/Indice}) \times \text{Elmt} \rightarrow \text{Liste}$
- `SupprimerAprès` : $\text{Liste} \times (\text{Adr/Indice}) \rightarrow \text{Liste} \times \text{Elmt}$
- `Rechercher` : $\text{Liste} \times \text{Elmt} \rightarrow (\text{Adr/Indice})$
- `Concaténer` : $\text{Liste} \times \text{Liste} \rightarrow \text{Liste}$



Impact du choix d'implémentation sur la complexité

Primitive	Tableau	Liste chaînée
Création	Taille bornée, espace perdu	
Insertion	En fin du tableau $O(1)$, mais $O(n)$ en position générale	Ajout d'un maillon : $O(1)$
Accès au k -ième terme	Immédiat : $O(1)$	Parcours de la liste nécessaire : $O(n)$
Suppression du k -ième terme	$O(1)$ seulement en fin de liste, sinon $O(n)$	$O(1)$ n'importe où si l'adresse du maillon précédent est connue, sinon parcours de la liste nécessaire : $O(n)$.

Stocker et retrouver un élément

Type abstrait de donnée Ensemble

Spécification informelle

Un **ensemble** est une collection qui permet de stocker des objets avec les opérations d'ajout, suppression et un test d'appartenance.



Type abstrait de donnée Ensemble

Spécification informelle

Un **ensemble** est une collection qui permet de stocker des objets avec les opérations d'ajout, suppression et un test d'appartenance.

Primitives souhaitées (et leur signature)

- `EnsblVide : () → Ensbl`.
- `EstVide : Ensbl → Booléen`
- `Insérer : Ensbl, Element → Ensbl`
- `Supprimer : Ensbl, Element → Ensbl`
- `Appartient : Ensbl, Element → Booléen`.



Type abstrait de donnée Ensemble

Spécification informelle

Un **ensemble** est une collection qui permet de stocker des objets avec les opérations d'ajout, suppression et un test d'appartenance.

Primitives souhaitées (et leur signature)

- `EnsblVide : () → Ensbl`.
- `EstVide : Ensbl → Booléen`
- `Insérer : Ensbl, Element → Ensbl`
- `Supprimer : Ensbl, Element → Ensbl`
- `Appartient : Ensbl, Element → Booléen`.



Exemples

Liste noire d'adresses IP. Données / sommets déjà traitées.

Spécification formelle du type abstrait Ensemble

Primitives souhaitées

- $\text{EnsemblVide} : () \rightarrow \text{Ensembl}$.
- $\text{EstVide} : \text{Ensembl} \rightarrow \text{Booléen}$
- $\text{Insérer} : \text{Ensembl}, \text{Element} \rightarrow \text{Ensembl}$
- $\text{Supprimer} : \text{Ensembl}, \text{Element} \rightarrow \text{Ensembl}$
- $\text{Appartient} : \text{Ensembl}, \text{Element} \rightarrow \text{Booléen}$.



Spécification

- La fonction EstVide doit satisfaire à

$$\text{EstVide}(e) = \begin{cases} \text{Vrai} & \text{si } e = \text{EnsembleVide}() \\ \text{Faux} & \text{si } e \text{ est de la forme } \text{Insérer}(e', y) \end{cases}$$

Spécification formelle du type abstrait Ensemble

Primitives souhaitées

- $\text{EnsemblVide} : () \rightarrow \text{Ensembl}$.
- $\text{EstVide} : \text{Ensembl} \rightarrow \text{Booléen}$
- $\text{Insérer} : \text{Ensembl}, \text{Element} \rightarrow \text{Ensembl}$
- $\text{Supprimer} : \text{Ensembl}, \text{Element} \rightarrow \text{Ensembl}$
- $\text{Appartient} : \text{Ensembl}, \text{Element} \rightarrow \text{Booléen}$.



Spécification

- La fonction Appartient satisfait :

$$\text{Appartient}(e, x) = \begin{cases} \text{Faux} & \text{si } e = \text{EnsembleVide}() \\ \text{Vrai} & \text{si } e \text{ est de la forme } \text{Insérer}(e', x) \\ \text{Appartient}(e', x) & \text{si } e \text{ est de la forme } \text{Insérer}(e', y) \end{cases}$$

Spécification formelle du type abstrait Ensemble

Primitives souhaitées

- $\text{EnsemblVide} : () \rightarrow \text{Ensembl}$.
- $\text{EstVide} : \text{Ensembl} \rightarrow \text{Booléen}$
- $\text{Insérer} : \text{Ensembl}, \text{Element} \rightarrow \text{Ensembl}$
- $\text{Supprimer} : \text{Ensembl}, \text{Element} \rightarrow \text{Ensembl}$
- $\text{Appartient} : \text{Ensembl}, \text{Element} \rightarrow \text{Booléen}$.



Spécification

- La fonction $\text{Supprimer}(e, x)$ n'est valide que si $\text{EstVide}(e)$ vaut Faux et alors :

$$\text{Supprimer}(\text{Insérer}(e, y), x) = \begin{cases} e & \text{si } x = y \\ \text{Insérer}(e', y) & \text{où } e' = \text{Supprimer}(e, x) \end{cases}$$

Type abstrait de donnée Dictionnaire

Spécification informelle

Une **table d'association**, ou **tableau associatif**, ou **dictionnaire** est une collection qui permet d'associer des valeurs à des clés, à raison d'au plus une valeur par clé.

Correspond à la notion de fonction en mathématique.



Type abstrait de donnée Dictionnaire

Spécification informelle

Une **table d'association**, ou **tableau associatif**, ou **dictionnaire** est une collection qui permet d'associer des valeurs à des clés, à raison d'au plus une valeur par clé.

Correspond à la notion de fonction en mathématique.

Primitives souhaitées (et leur signature)

- `DicoVide : () → Dico.`
- `EstVide : Dico → Booléen`
- `Insérer : Dico, Clé, Valeur → Dico`
- `Supprimer : Dico, Clé → Dico`
- `Rechercher : Dico, Clé → Valeur ∪ Erreur.`



Type abstrait de donnée Dictionnaire

Spécification informelle

Une **table d'association**, ou **tableau associatif**, ou **dictionnaire** est une collection qui permet d'associer des valeurs à des clés, à raison d'au plus une valeur par clé.

Correspond à la notion de fonction en mathématique.

Primitives souhaitées (et leur signature)

- `DicoVide : () → Dico`.
- `EstVide : Dico → Booléen`
- `Insérer : Dico, Clé, Valeur → Dico`
- `Supprimer : Dico, Clé → Dico`
- `Rechercher : Dico, Clé → Valeur ∪ Erreur`.



Exemples

Gestion des noms de variables ou de fichiers. Création de mémoire virtuelle. Stockage de mots de passe.

Spécification formelle du type abstrait Dictionnaire

Primitives souhaitées

- $\text{DicoVide} : () \rightarrow \text{Dico}$.
- $\text{EstVide} : \text{Dico} \rightarrow \text{Booléen}$
- $\text{Insérer} : \text{Dico}, \text{Clé}, \text{Valeur} \rightarrow \text{Dico}$
- $\text{Supprimer} : \text{Dico}, \text{Element} \rightarrow \text{Dico}$
- $\text{Rechercher} : \text{Dico}, \text{Clé} \rightarrow \text{Valeur} \cup \text{Erreur}$.



Spécification

- La fonction EstVide doit satisfaire à

$$\text{EstVide}(d) = \begin{cases} \text{Vrai} & \text{si } d = \text{DicoVide}() \\ \text{Faux} & \text{si } d \text{ est de la forme } \text{Insérer}(d', x, v) \end{cases}$$

Spécification formelle du type abstrait Dictionnaire

Primitives souhaitées

- $\text{DicoVide} : () \rightarrow \text{Dico}$.
- $\text{EstVide} : \text{Dico} \rightarrow \text{Booléen}$
- $\text{Insérer} : \text{Dico}, \text{Clé}, \text{Valeur} \rightarrow \text{Dico}$
- $\text{Supprimer} : \text{Dico}, \text{Element} \rightarrow \text{Dico}$
- $\text{Rechercher} : \text{Dico}, \text{Clé} \rightarrow \text{Valeur} \cup \text{Erreur}$.



Spécification

- La fonction Rechercher satisfait :

$$\text{Rech.}(d, x) = \begin{cases} \text{Erreur} & \text{si } e = \text{DicoVide}() \\ v & \text{si } d \text{ est de la forme } \text{Insérer}(d', x, v) \\ \text{Rech.}(d', x) & \text{si } d \text{ est de la forme } \text{Insérer}(d', y) \\ & \text{où } x \neq y \end{cases}$$

Spécification formelle du type abstrait Dictionnaire

Primitives souhaitées

- $\text{DicoVide} : () \rightarrow \text{Dico}$.
- $\text{EstVide} : \text{Dico} \rightarrow \text{Booléen}$
- $\text{Insérer} : \text{Dico}, \text{Clé}, \text{Valeur} \rightarrow \text{Dico}$
- $\text{Supprimer} : \text{Dico}, \text{Element} \rightarrow \text{Dico}$
- $\text{Rechercher} : \text{Dico}, \text{Clé} \rightarrow \text{Valeur} \cup \text{Erreur}$.



Spécification

- La fonction $\text{Supprimer}(e, x)$ n'est valide que si $\text{EstVide}(e)$ vaut Faux et alors :

$$\text{Suppr.}(\text{Insérer}(d, y, v), x) = \begin{cases} d & \text{si } x = y \\ \text{Insérer}(d', y, v) & \text{où } d' = \text{Suppr.}(d, x) \end{cases}$$

Parenté entre ensembles et dictionnaire

Si d est de type « dictionnaire »,
alors en n'insérant que des
associations

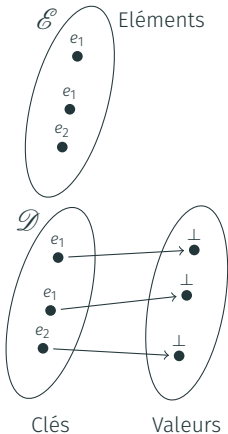
Clé $\mapsto$ **None**

on retrouve le type « Ensemble ».

Inversement, on peut souvent
dédurre un dictionnaire d'une
structure de données d'ensemble.

Conséquence

Nous étudions seulement des
structures de données
implémentant des ensembles.



Implémentation par un tableau de booléens

Principe

On numérote les éléments possibles. Un tableau contient un booléen qui indique si un élément est présent ou non.

Implémentation par un tableau de booléens

Principe

On numérote les éléments possibles. Un tableau contient un booléen qui indique si un élément est présent ou non.

Exemple

Ensemble $E = \{1, 4, 7, 9\}$.

F	V	F	F	V	F	F	V	F	V
E[0]	E[1]	E[2]	E[3]	E[4]	E[5]	E[6]	E[7]	E[8]	E[9]

Implémentation par un tableau de booléens

Principe

On numérote les éléments possibles. Un tableau contient un booléen qui indique si un élément est présent ou non.

Exemple

Ensemble $E = \{1, 4, 7, 9\}$.

F	V	F	F	V	F	F	V	F	V
E[0]	E[1]	E[2]	E[3]	E[4]	E[5]	E[6]	E[7]	E[8]	E[9]

Avantages et inconvénients

- Demande de la place en mémoire, mais souvent implémentation spéciale (différente des tableaux) pour stocker l'information à l'échelle du bit.
- Insérer, supprimer et appartient sont rapides $O(1)$.
Intersection ou union revient à calculer AND et OR bit à bit.
- Applications : index pour optimiser des recherches dans des bases de données.

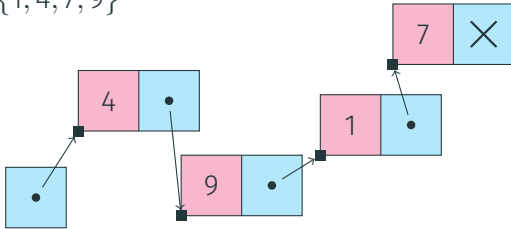
Implémentation par une liste ou un tableau

Principe

On stocke les éléments dans une liste ou un tableau

Exemple

Ensemble $E = \{1, 4, 7, 9\}$



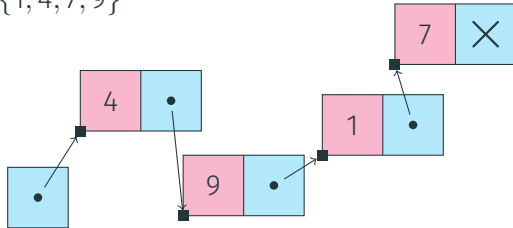
Implémentation par une liste ou un tableau

Principe

On stocke les éléments dans une liste ou un tableau

Exemple

Ensemble $E = \{1, 4, 7, 9\}$



Avantages et inconvénients

- Simple
- Opérations lentes (complexité linéaire pour trouver un élément) si l'ensemble est grand.
- Mais très rapides si l'ensemble est petit.

Implémentation par un tableau trié

Principe

On stocke les éléments dans un tableau toujours trié.

Implémentation par un tableau trié

Principe

On stocke les éléments dans un tableau toujours trié.

Exemple

Ensemble $E = \{1, 4, 7, 9\}$

4

longueur

1	4	7	9	931	667	207	175	446
T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]

T

Implémentation par un tableau trié

Principe

On stocke les éléments dans un tableau toujours trié.

Exemple

Ensemble $E = \{1, 4, 7, 9\}$

4

1	4	7	9	931	667	207	175	446
T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]

longueur

T

Avantages et inconvénients

- Nécessite un ordre sur les éléments
- La recherche d'un élément est de complexité logarithmique :
recherche dichotomique.
- Insertion, suppression difficiles (complexité linéaire).

Définition inductive des arbres binaires de recherche

Définition

Soit $(E, \leq)$ un ensemble ordonné.

L'ensemble $\mathcal{A}_{ABR}$ des **arbres binaires de recherche** est la partie des arbres binaire étiquetés dans E définie inductivement par les règles d'inférence suivante :

$$\frac{}{\perp \in \mathcal{A}}$$

et

$$\frac{g \in \mathcal{A}_{ABR} \quad d \in \mathcal{A}_{ABR} \quad \begin{cases} g = \perp \text{ ou } \max(g) \leq e \text{ et} \\ d = \perp \text{ ou } e \leq \min(d) \end{cases}}{(e|g|d) \in \mathcal{A}_{ABR}}$$

où e appartient à E .

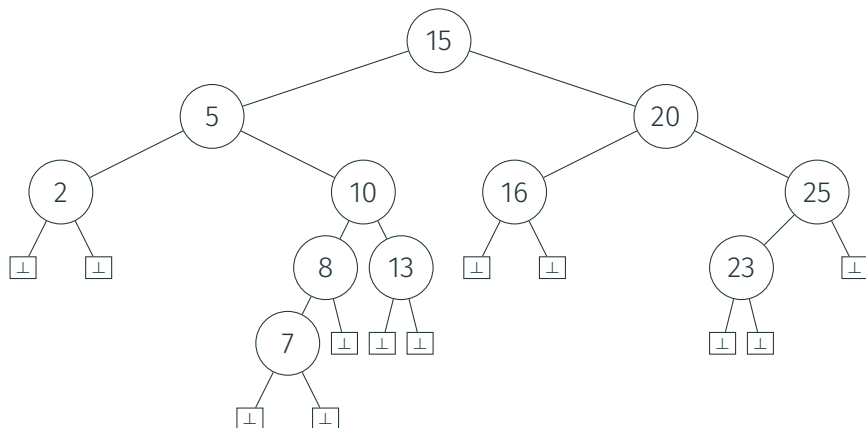
Remarque

$\mathcal{A}_{ABR}$ représente des multiensembles.

Rendre les inégalités strictes pour interdire les doublons.

Illustration : arbre binaire de recherche

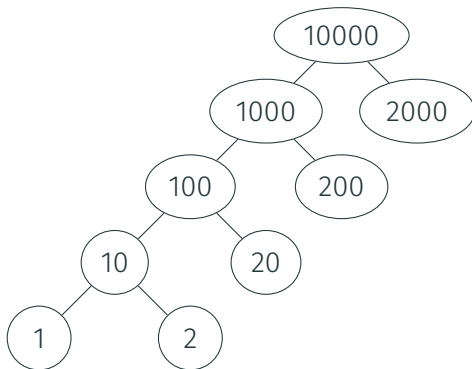
Exemple



Quiz : arbre binaire de recherche

Vrai ou faux

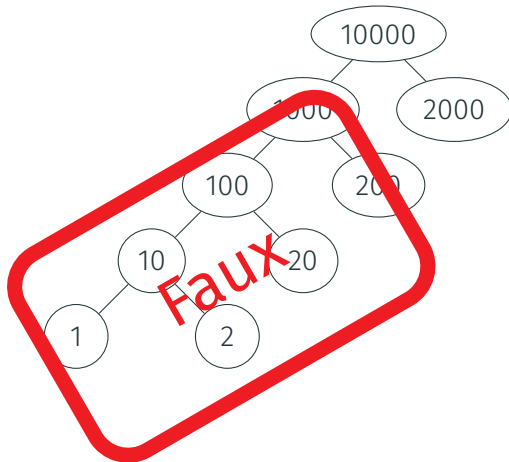
L'arbre suivant est un arbre binaire de recherche



Quiz : arbre binaire de recherche

Vrai ou faux

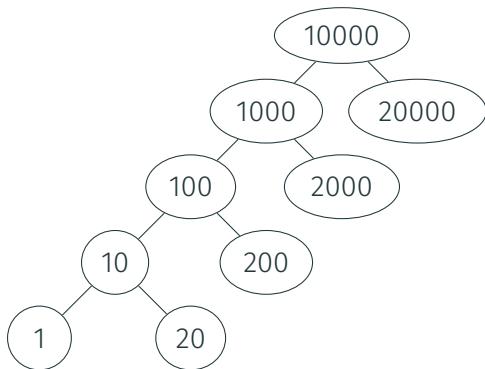
L'arbre suivant est un arbre binaire de recherche



Quiz : arbre binaire de recherche

Vrai ou faux

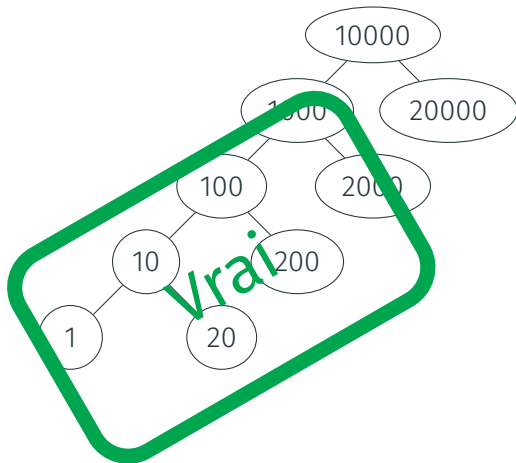
L'arbre suivant est un arbre binaire de recherche



Quiz : arbre binaire de recherche

Vrai ou faux

L'arbre suivant est un arbre binaire de recherche



Appartenance à un ABR

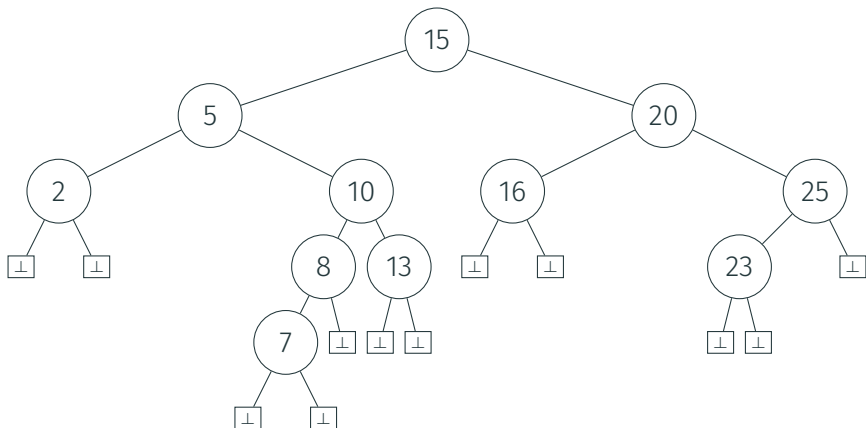
Pour définir $\text{Appartient}(a, x)$, on exploite la structure inductive :

et

$$\frac{\text{Appartient}(\perp, x) = \text{F}}{g \in \mathcal{A}_{\text{ABR}} \quad d \in \mathcal{A}_{\text{ABR}} \quad \left\{ \begin{array}{l} g = \perp \text{ ou } \max(g) \leq e \text{ et} \\ d = \perp \text{ ou } e \leq \min(d) \end{array} \right.}$$
$$\text{Appartient}((e|g|d), x) = \begin{cases} \text{V} & \text{si } x = e \\ \text{Appartient}(g, x) & \text{si } x < e \\ \text{Appartient}(d, x) & \text{si } x > e \end{cases}$$

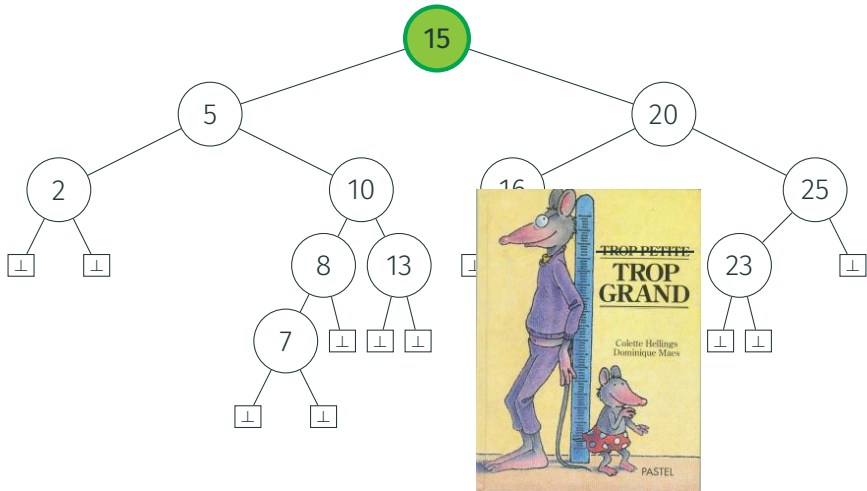
Primitive Appartient : illustration

Exemple (7 appartient-il à l'ABR?)



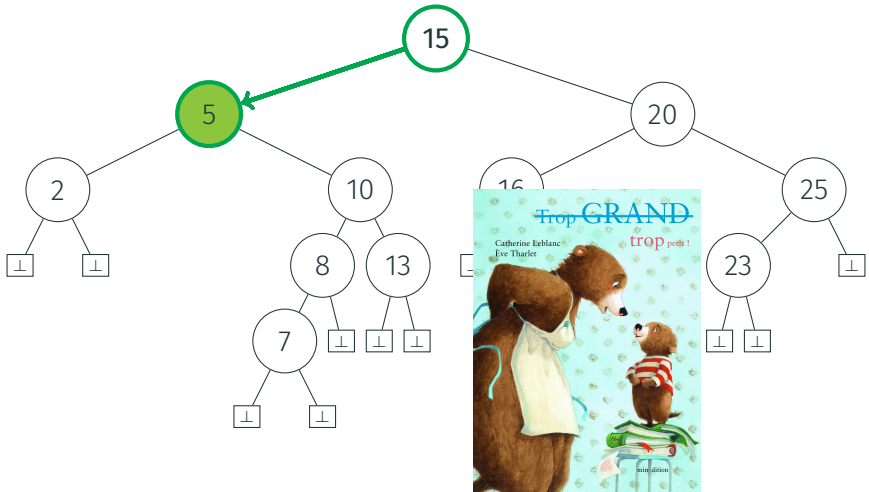
Primitive Appartient : illustration

Exemple (7 appartient-il à l'ABR?)



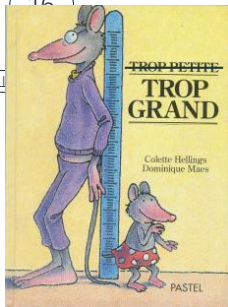
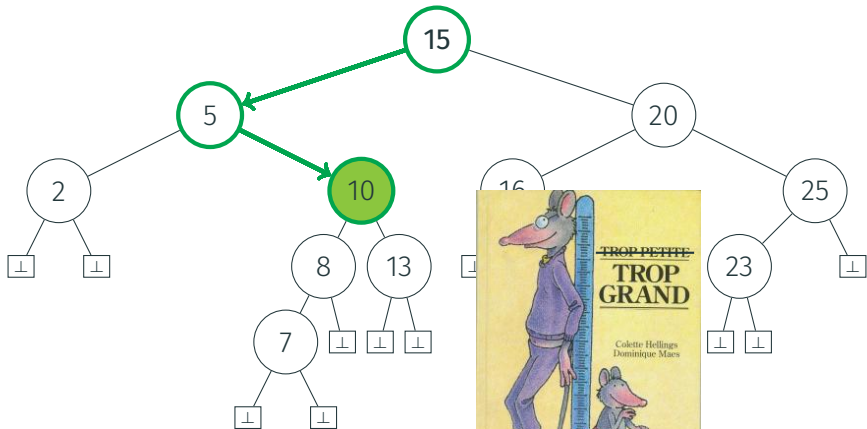
Primitive Appartient : illustration

Exemple (7 appartient-il à l'ABR ?)



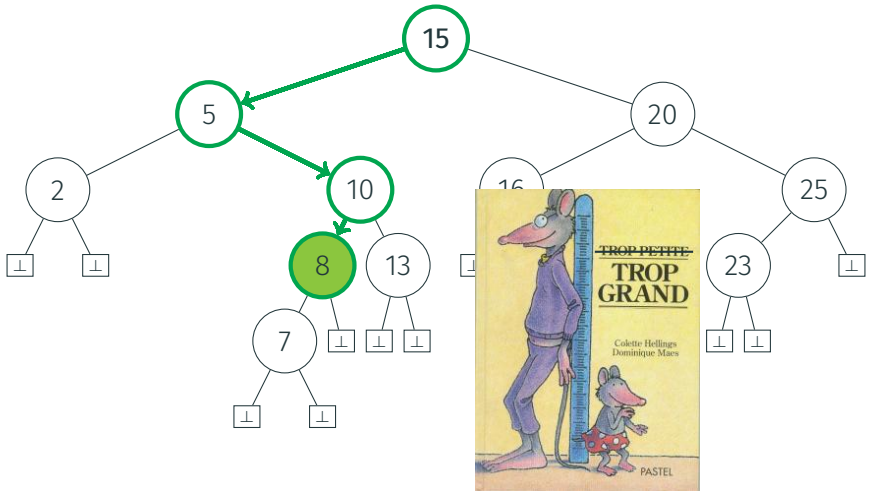
Primitive Appartient : illustration

Exemple (7 appartient-il à l'ABR?)



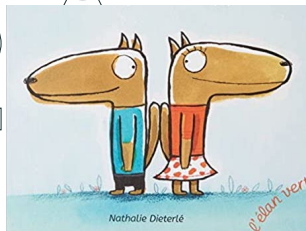
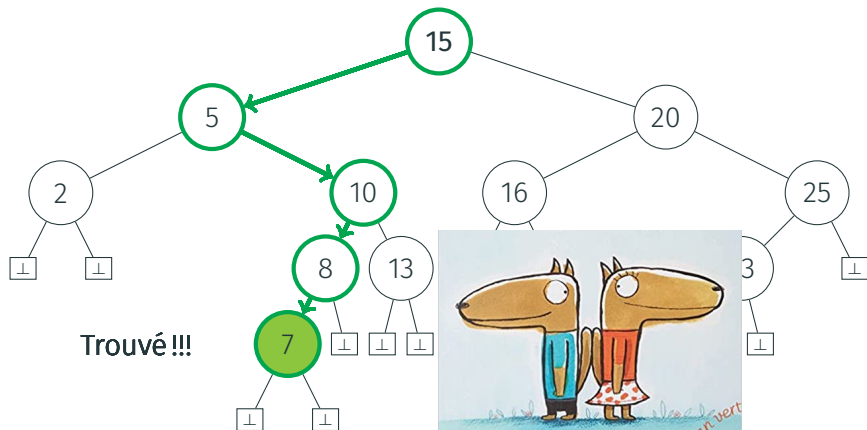
Primitive Appartient : illustration

Exemple (7 appartient-il à l'ABR?)



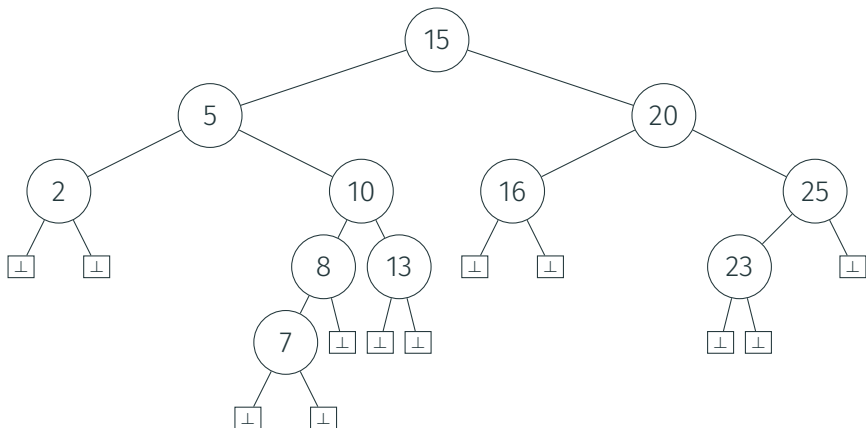
Primitive Appartient : illustration

Exemple (7 appartient-il à l'ABR?)



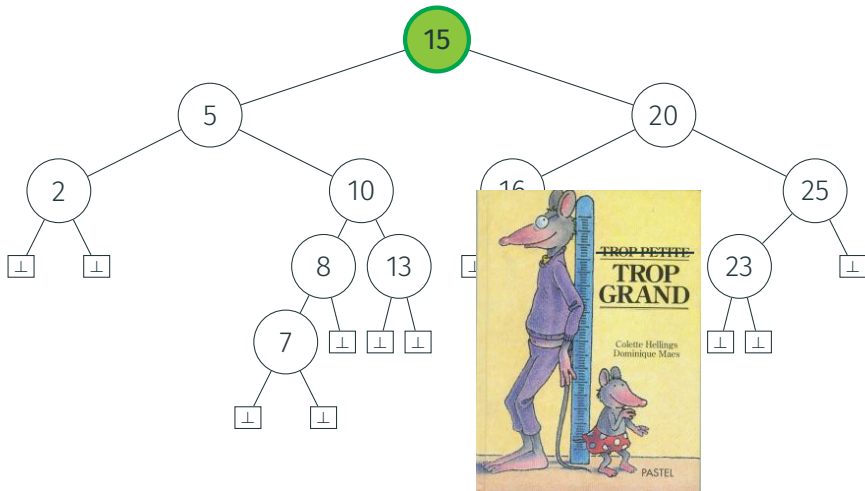
Primitive Appartient : illustration

Exemple (14 appartient-il à l'ABR?)



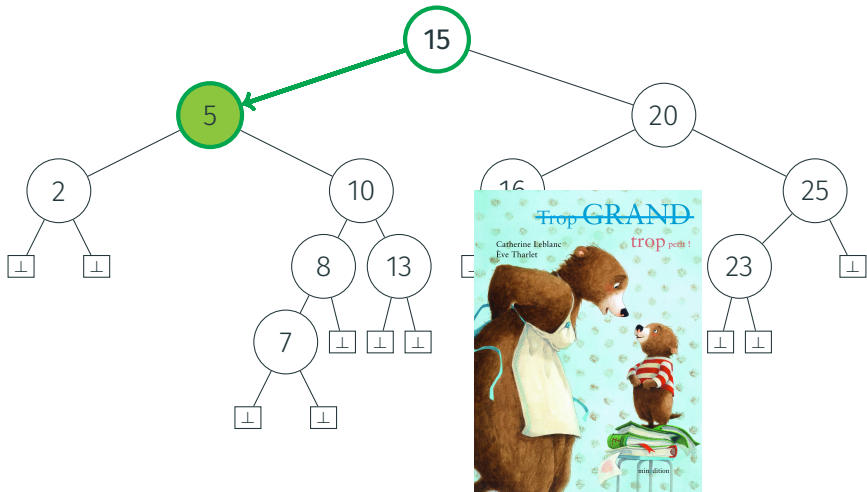
Primitive Appartient : illustration

Exemple (14 appartient-il à l'ABR?)



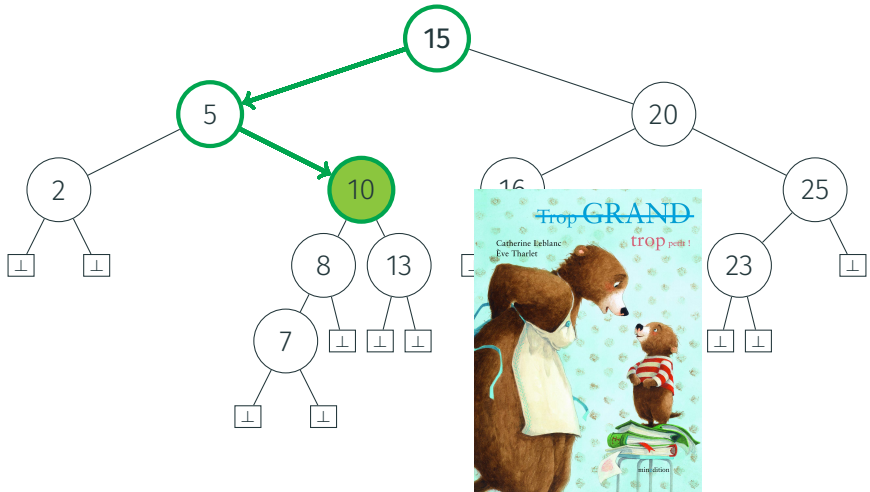
Primitive Appartient : illustration

Exemple (14 appartient-il à l'ABR?)



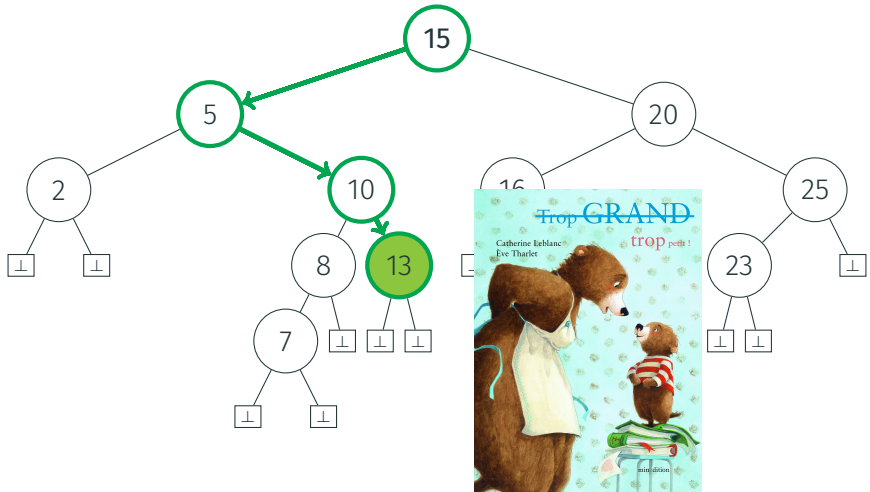
Primitive Appartient : illustration

Exemple (14 appartient-il à l'ABR?)



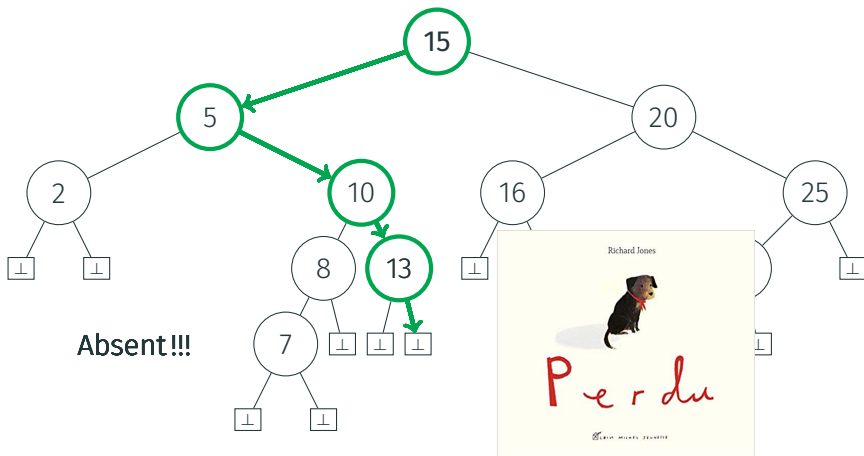
Primitive Appartient : illustration

Exemple (14 appartient-il à l'ABR?)



Primitive Appartient : illustration

Exemple (14 appartient-il à l'ABR?)



Appartenance à un ABR en Python

```
class ABR():
    def __init__(self, d, a = None, b = None):
        self.data = d
        self.left = a
        self.right = b

def find(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return find(root.left, key)
    else:
        return find(root.right, key)
```

Insertion dans un ABR

Pour définir **Insérer**(a, x), on exploite la structure inductive :

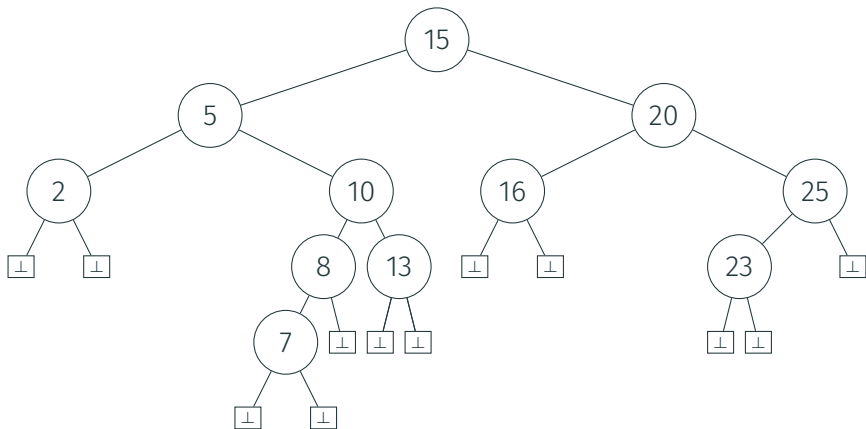
$$\overline{\text{Insérer}(\perp, x) = (x | \perp | \perp)}$$

et

$$\frac{g \in \mathcal{A}_{\text{ABR}} \quad d \in \mathcal{A}_{\text{ABR}} \quad \begin{cases} g = \perp \text{ ou } \max(g) \leq e \text{ et} \\ d = \perp \text{ ou } e \leq \min(d) \end{cases}}{\text{Insérer}((e | g | d), x) = \begin{cases} (e | g | d) & \text{si } x = e \\ (e | \text{Insérer}(g, x) | d) & \text{si } x < e \\ (e | g | \text{Insérer}(d, x)) & \text{si } x > e \end{cases}}$$

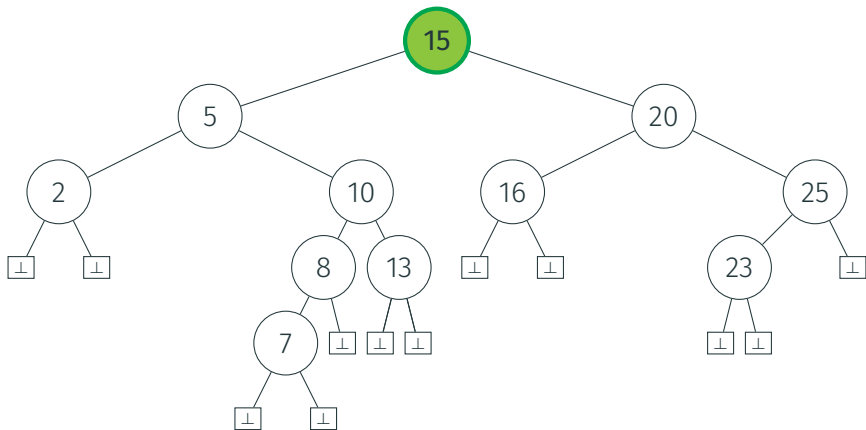
Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



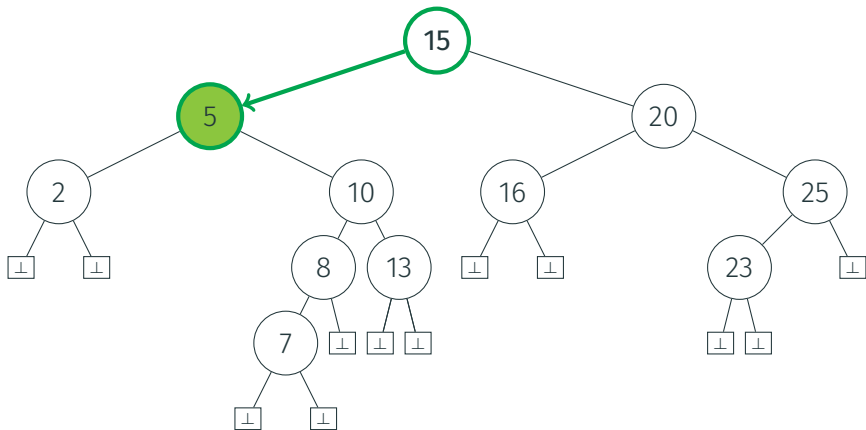
Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



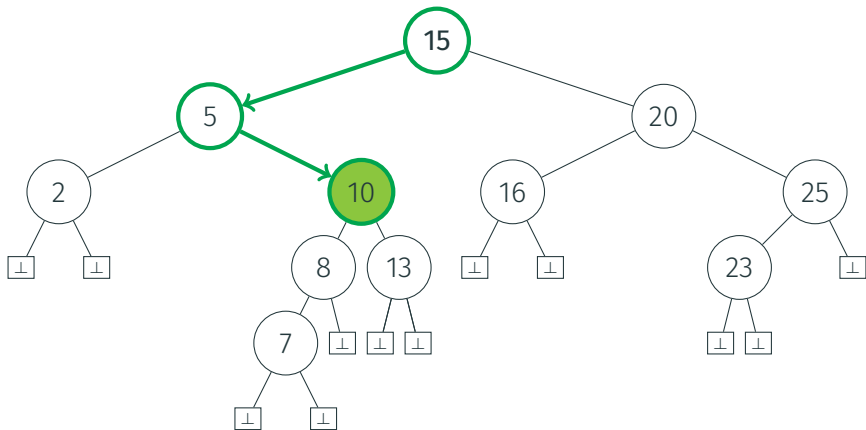
Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



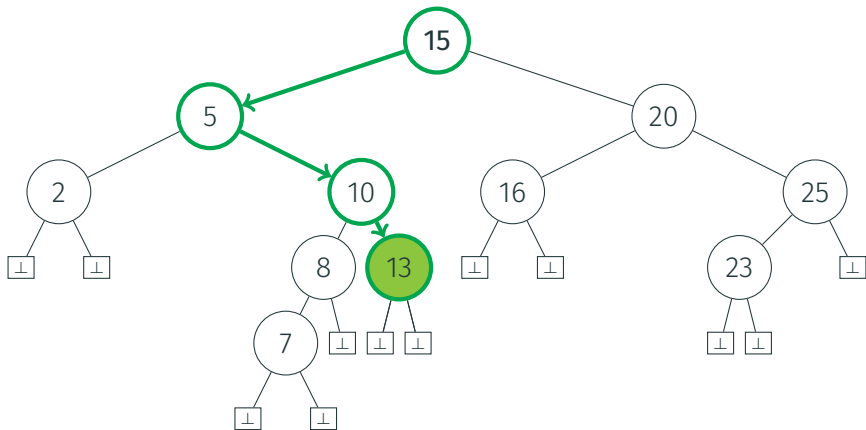
Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



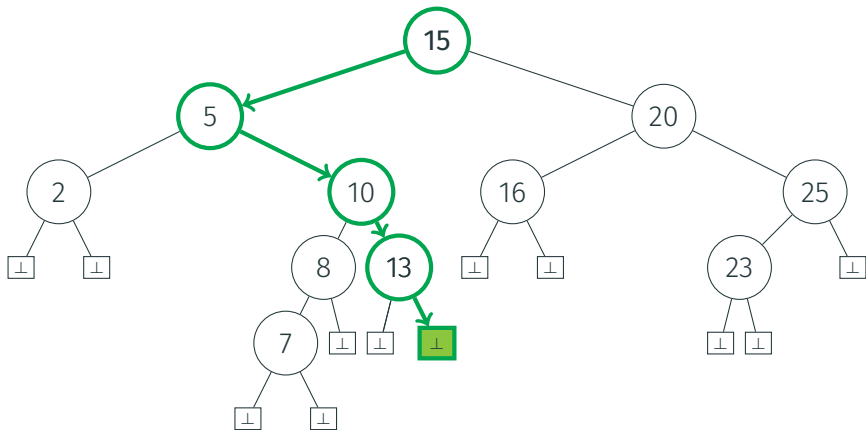
Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



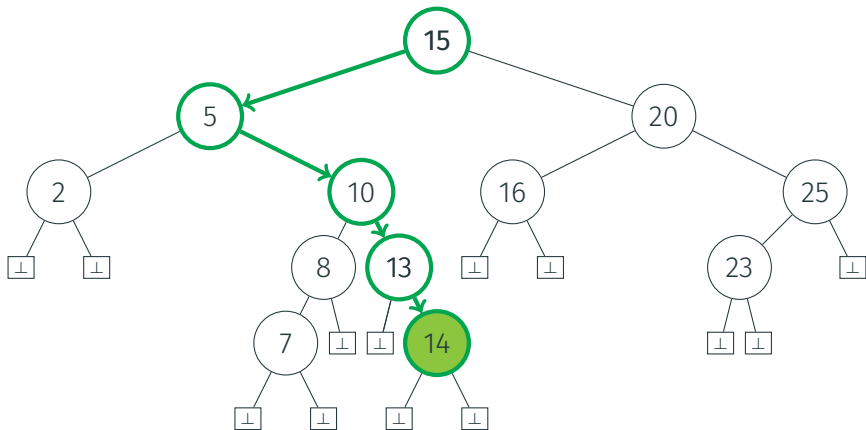
Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



Primitive Insérer : illustration

Exemple (Comment insérer 14 à l'ABR?)



Insérer dans un ABR en Python

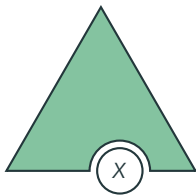
```
class ABR():
    def __init__(self, d, a = None, b = None):
        self.data = d
        self.left = a
        self.right = b

def insert(root, key):
    if root is None:
        return ABR(key)
    if key < root.data:
        root.left = insert(root.left, key)
    elif key > root.data:
        root.right = insert(root.right, key)
    return root
```


Primitive Supprimer : principe

Cas 0 : supprimer un élément x sans fils

- On peut supprimer le sommet concerné sans problème



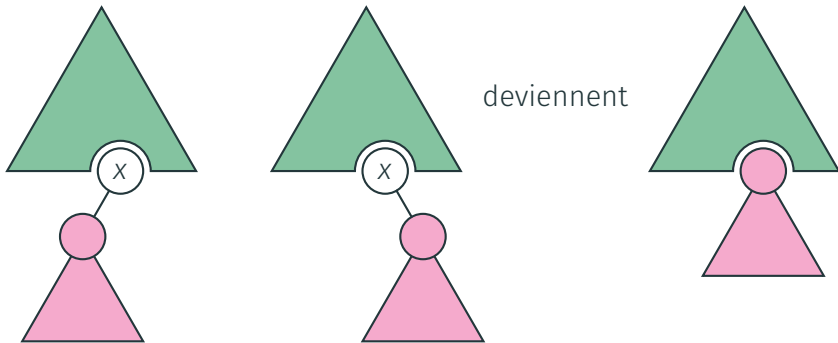
devient



Primitive Supprimer : principe

Cas 1 : supprimer un élément x possédant un seul fils

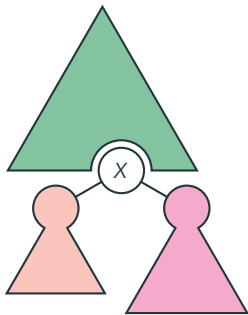
- On raccourcit l'arbre au niveau du sommet supprimé



Primitive Supprimer : principe

Cas 2 : supprimer un élément x possédant deux fils

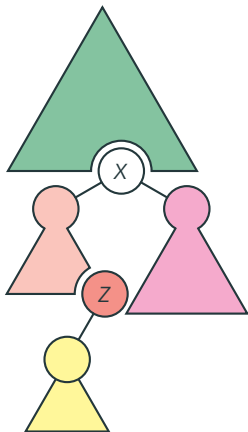
- Des idées? $\neg_(_)_/\neg$



Primitive Supprimer : principe

Cas 2 : supprimer un élément x possédant deux fils

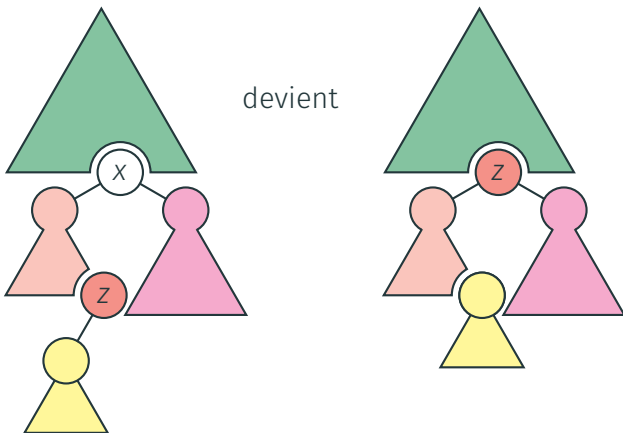
- On cherche le **maximum** dans le sous-arbre gauche de x , qui est le dernier élément rencontré en suivant la suite des fils droits.



Primitive Supprimer : principe

Cas 2 : supprimer un élément x possédant deux fils

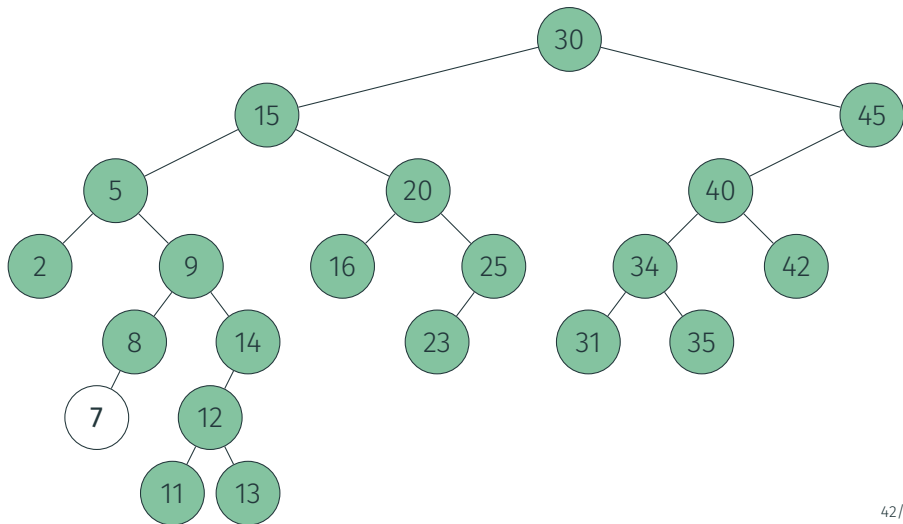
- On remplace x par z et on supprime z .



Primitive supprimer : illustration cas 0

Exemple

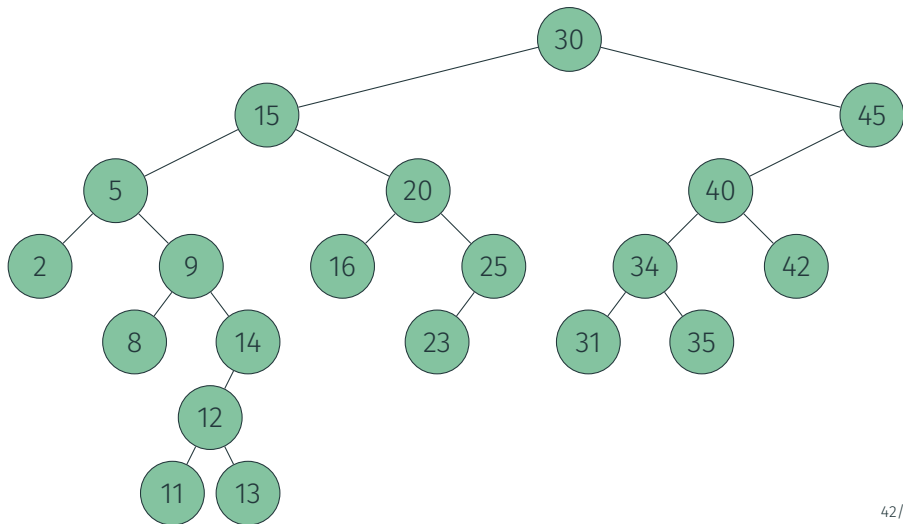
Supprimer 7



Primitive supprimer : illustration cas 0

Exemple

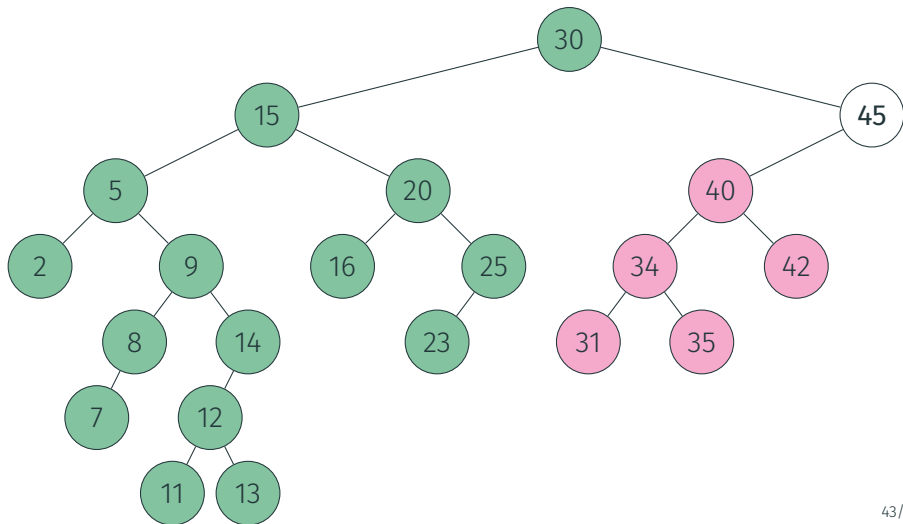
Supprimer 7



Primitive suppresser : illustration cas 1

Exemple

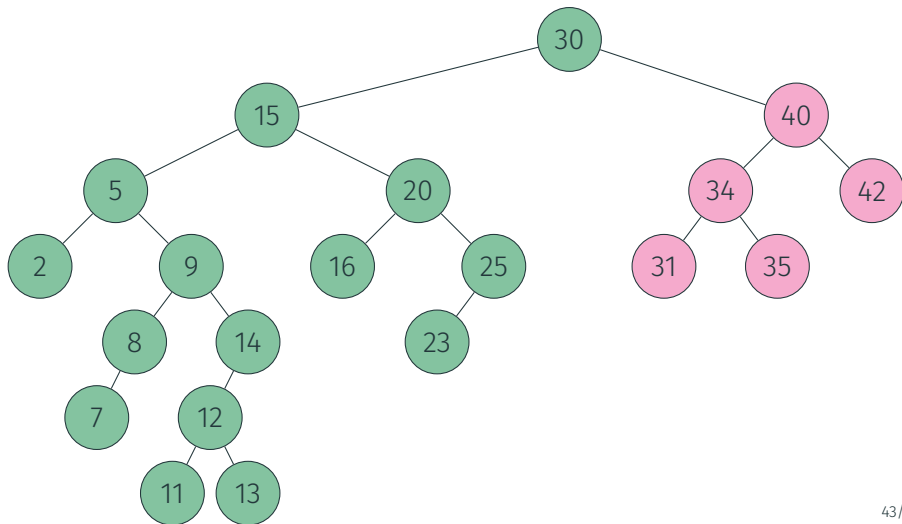
Supprimer 45



Primitive suppresser : illustration cas 1

Exemple

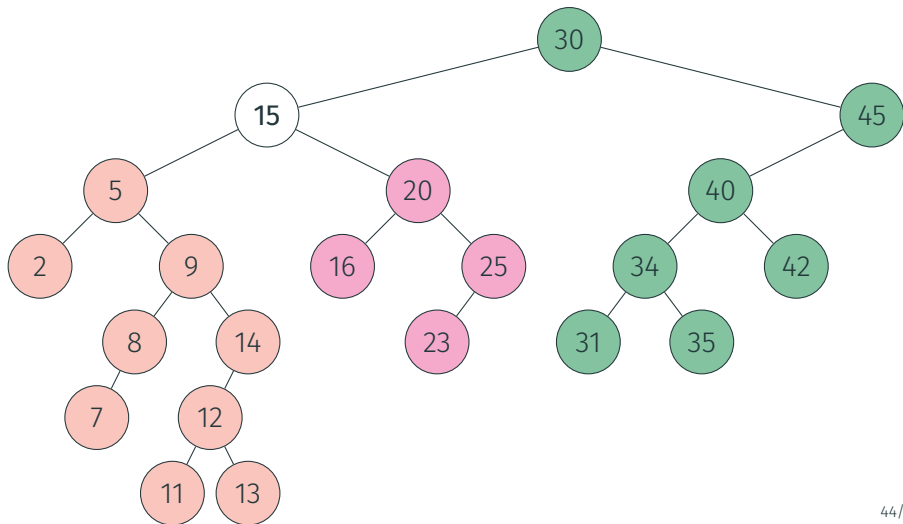
Supprimer 45



Primitive suppresser : illustration cas 2

Exemple

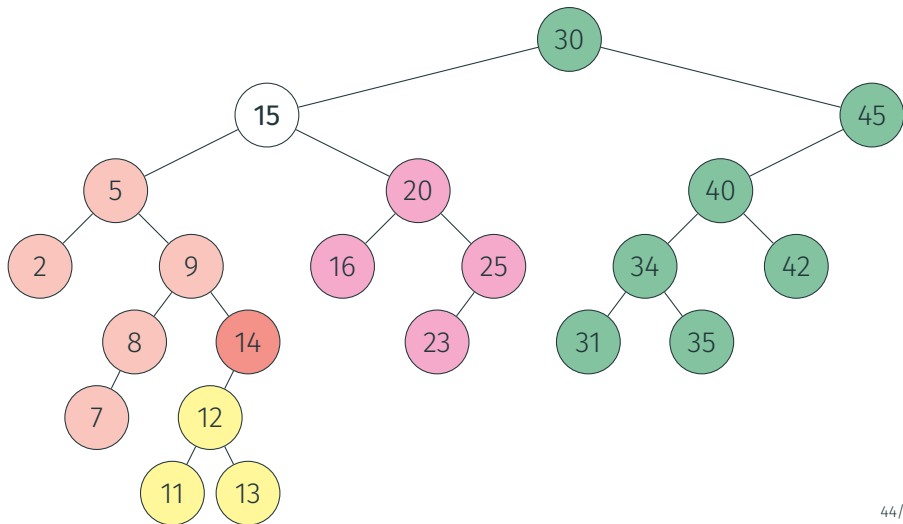
Supprimer 15



Primitive suppresser : illustration cas 2

Exemple

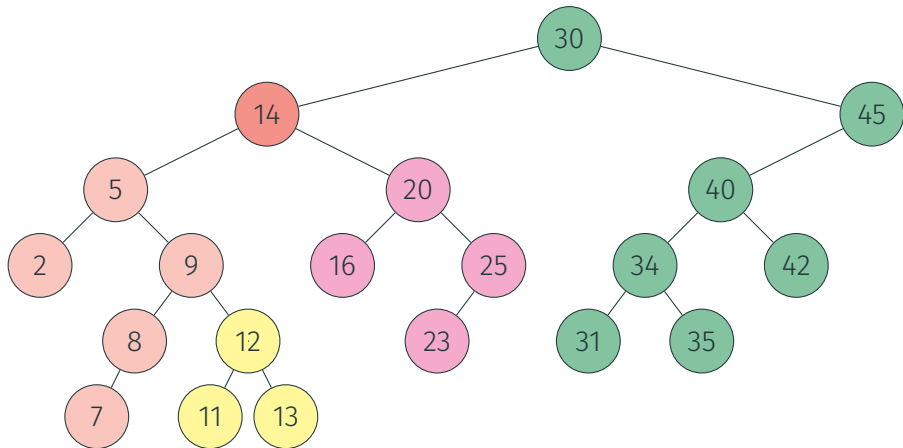
Supprimer 15



Primitive suppresser : illustration cas 2

Exemple

Supprimer 15



Supprimer dans un ABR en Python

```
def find_max(root):
    if root is None or root.right is None:
        return root
    else:
        return find_max(root.right)

def remove(root, key):
    if root is None:
        pass
    elif root.data > key:
        root.left = remove(root.left, key)
    elif root.data < key:
        root.right = remove(root.right, key)
    else:
        if root.left is None:
            root = root.right
        elif root.right is None:
            root = root.left
        else:
            mnode = find_min(root.right)
            root.data = mnode.data
            root.right = remove(root.right, root.data)
    return root
```

Supprimer dans un ABR (vue fonctionnelle)

D'une part

$$\begin{array}{c}
 \text{Suppr_max}(\perp, x) = \text{Erreur} \\
 \text{et} \\
 \frac{g \in \mathcal{A}_{\text{ABR}} \quad d \in \mathcal{A}_{\text{ABR}} \quad \begin{cases} g = \perp \text{ ou } \max(g) \leq e \text{ et} \\ d = \perp \text{ ou } e \leq \min(d) \end{cases}}{\text{Suppr_max}((e|g|d), x) = \begin{cases} e, g & \text{si } d = \perp \\ m, (e|g|d') & \text{où } (m, d') = \text{Suppr_max}(d, x) \end{cases}}
 \end{array}$$

D'autre part,

$$\begin{array}{c}
 \text{Suppr}(\perp, x) = \perp \\
 \text{et} \\
 \frac{g \in \mathcal{A}_{\text{ABR}} \quad d \in \mathcal{A}_{\text{ABR}} \quad \begin{cases} g = \perp \text{ ou } \max(g) \leq e \text{ et} \\ d = \perp \text{ ou } e \leq \min(d) \end{cases}}{\text{Suppr}((e|g|d), x) = \begin{cases} g & \text{si } x = e \text{ et } d = \perp \\ d & \text{si } x = e \text{ et } g = \perp \\ (m|g'|d) & \text{si } x = e \text{ et } (m, g') = \text{Suppr_max}(g) \\ (e|\text{Suppr}(g, x)|d) & \text{si } x < e \\ (e|g|\text{Suppr}(d, x)) & \text{si } x > e \end{cases}}
 \end{array}$$

Remarque

Le parcours en ordre **infixe** d'un ABR restitue l'ensemble des éléments par ordre croissant.

Application

En rangeant des éléments dans un ABR, on peut

- trier les éléments
- construire facilement un itérateur qui parcourt la collection par ordre croissant.

Complexité

Dans un ABR à n sommets,

	Pire des cas	Moyenne
Insérer	$O(n)$	$O(\log n)$
Supprimer	$O(n)$	$O(\log n)$
Chercher	$O(n)$	$O(\log n)$

Esquisse de preuve

La complexité est dominée par les recherches d'éléments, de l'ordre de $O(h)$ où h est la hauteur de l'arbre.

On peut démontrer que si A est l'arbre obtenu après insertion de n éléments dans l'ordre d'une permutation aléatoire tirée selon la loi uniforme, alors

$$\mathbb{E}[h] = \Theta(\log n).$$

Amélioration des complexités des ABR

Des techniques supplémentaires (**arbres bicolores** ou **arbres AVL**) permettent de forcer l'ABR à rester de hauteur $\Theta(\log n)$ en procédant à des rééquilibrages entre branches si besoin.

Avantage

Toutes les complexités sont $O(\log n)$ dans le pire des cas.

Inconvénient

L'implémentation de la structure est plus compliquée.

~1978



Robert Sedgwick
(1946)

~1962



Georgy Adelson-Velsky
(1922+2014)



Evgenii Landis
(1921+1997)